Describe Your Solution — text to paste into the form

Who Sees What · Twinforce Solutions Limited · version 0.3.0.1 · 17 August 2026

Copy each block below into the matching field. Nothing here needs editing.

Four of the six fields should have their toggle set to **Disabled**, because they do not apply to this package. Answers are supplied anyway in case the form insists on text.

---

1. Detailed technical description of your solution and related components

Toggle: n/a — this field is always required.

Who Sees What answers one question for a Salesforce administrator: what can this user see and do, and what gave them that access?

An admin picks a user, then works from apps to objects to fields. For each object we report create, read, update, delete, view all and modify all. For each field we report whether it is readable and editable. Beside every grant we show which permission set, permission set group, or profile produced it.

The tool is read-only. It grants nothing, changes no permissions, and writes nothing except its own audit log.

The whole package runs on the Salesforce Platform. There is no web application, no external service, and no callout of any kind. It contains:

- 16 Apex classes and 6 Apex test classes, all declared `with sharing`
- 22 Lightning Web Component bundles
- One custom object, PE_Inspection_Log__c, with five fields
- One custom permission, Use_Permission_Explorer
- One permission set, PermissionCheckerPermissionSet
- One Lightning Component tab and one Lightning app
- One Platform Cache partition, PermissionExplorer

It is a second-generation managed package in the twinforce namespace, built against API version 67.0, containing 165 metadata files.

How access is controlled. Every @AuraEnabled method on PermissionExplorerController starts by calling PEAccessGate.assertAuthorised(). That method checks the packaged custom permission Use_Permission_Explorer through FeatureManagement.checkPermission and throws before any query runs if the caller does not hold it. Two tests assert this directly: PermissionExplorerControllerTest.unauthorisedCallerIsRejectedByEveryEntryPoint walks every entry point, and PEUserDetailServiceTest.unauthorisedUserIsRejectedBeforeAnyQuery confirms the rejection happens before any SOQL executes.

The custom permission is granted only through the packaged permission set. We do not auto-assign it on install. Assigning it lets someone inspect every user in the org, so we think that should be a decision an administrator makes on purpose, not something the installer does quietly on their behalf. The in-app onboarding panel says exactly that.

How permissions are enforced. All 16 implementation classes are `public with sharing`. Every query runs in user mode — either `WITH USER_MODE` on inline SOQL, or Database.queryWithBinds with AccessLevel.USER_MODE for the one query we assemble at runtime. So the platform enforces object and field permissions for whoever is running the tool. An admin cannot use it to see permission data they are not themselves entitled to read.

What we read. Only permission metadata and user records: User, UserLogin, UserRole, Profile, PermissionSet, PermissionSetAssignment, PermissionSetGroup, PermissionSetGroupComponent, PermissionSetTabSetting, ObjectPermissions, FieldPermissions, LoginHistory, AppMenuItem, TabDefinition, EntityDefinition.

We read no business data. We read the permission rows that describe objects like Account, never Account records themselves.

What we write. One row per inspection to PE_Inspection_Log__c, recording who looked, who was looked at, what they did, and when. The packaged permission set grants Read and Create on that object and deliberately withholds Edit and Delete, so a holder can review the log but cannot alter or erase their own trail.

One dynamically assembled query. PEUserSearchService.search builds a SOQL string at runtime to apply optional filters. It is assembled only from private static final constants; filters are applied by choosing which constant to append, never by concatenating a value. Every value reaches the database through a named bind map. The one identifier that varies is the sort column, and that comes from an allowlist in PEKeysetCursor.isAllowedSort, after which the sort is applied by picking between two constant ORDER BY fragments. Two tests cover this: PEUserSearchServiceTest.quoteInTermIsBoundNotInterpolated and PermissionExplorerTest.disallowedSortFieldIsRejected.

Where we stop. Per-user record type visibility and per-permission-set app assignment live only inside Profile and PermissionSet metadata XML, which is reachable only through the Metadata or Tooling API — a callout. We chose not to make callouts, so the interface reports those two areas as not analysed rather than guessing. Login history needs the Manage Users permission, which we do not grant, so where the running admin lacks it the panel explains that instead of showing an empty list. We treat these disclosures as part of the product: an admin who mistakes “not analysed” for “no access” would make a worse decision than one who is told where the answer stops.

74 Apex tests ship with the package. All pass. Code coverage is 88%.

---

2. If your solution includes or requires a web app or service, list the frameworks and languages that it uses

Toggle: Disabled.

The package needs no web application or web service. Everything is Apex and Lightning Web Components running on the Salesforce Platform.

---

3. If your solution uses any platforms other than Salesforce Platform, list them and describe how they're used

Toggle: Disabled.

We use no other platform. There is no Heroku component, no Marketing Cloud component, and nothing hosted outside Salesforce.

---

4. If your solution integrates with technology outside the Salesforce Platform, provide details

Toggle: Disabled.

There are no integrations outside the platform. The package contains no HTTP callouts, no Named Credentials, no External Credentials, and no Remote Site Settings. Searching the package source for HttpRequest, new Http(), and callout: returns zero matches in all three cases.

---

5. If your solution contains Salesforce Platform technology, such as Lightning Components and Apex, provide details

Toggle: Enabled.

Apex. 16 implementation classes and 6 test classes, all public with sharing. The entry points are the @AuraEnabled methods on PermissionExplorerController: searchUsers, resolveUser, getAppObjects, getObjectDetail, getFilterOptions, getLoginHistory, getAssignedPermissionSets, getAssignedPermissionSetGroups, and refreshCache. Each begins with PEAccessGate.assertAuthorised(). All SOQL runs in user mode.

Lightning Web Components. 22 bundles. Only permissionExplorer is exposed; it is surfaced as a Lightning Component tab. The other 21 are internal children with isExposed set to false. No component holds credentials or makes network calls of its own — all server communication goes through the Apex methods above.

Custom object. PE_Inspection_Log__c, append-only, five fields: Action, Detail, Inspected At, Inspector User, Subject User.

Custom permission. Use_Permission_Explorer, the only authorisation gate.

Platform Cache. Partition PermissionExplorer, 2 MB org scope and 1 MB session scope. We cache resolved permission results for 15 minutes and describe metadata for an hour, both keyed per user. Nothing is cached that the running admin could not already see. Where a subscriber org has no cache allocation, PECacheManager falls back to a transaction-scoped map, so the tool works correctly without cache — that path is exercised by the test suite, not just compiled.

Not present: Visualforce, Apex triggers, Flows, Aura components, batch or scheduled jobs, Platform Events, custom settings.

---

6. If your solution stores any Salesforce OAuth tokens or other access tokens, describe how the tokens are stored

Toggle: Disabled.

We store no tokens of any kind — no OAuth tokens, session tokens, API keys, or passwords. There is no authentication flow in the package; it runs inside the subscriber's existing Salesforce session. There are no Named Credentials, External Credentials, Auth Providers, or Connected Apps, and no field on any object or setting holds credential data.

---

7. If your solution stores any Salesforce data outside the Salesforce Platform, list all types of stored data, and describe where and how the data is stored

Toggle: Disabled.

No Salesforce data leaves the platform. We transmit nothing off-platform. Everything the package writes stays in the subscriber's own org: audit rows in PE_Inspection_Log__c, and cached values in the subscriber's Platform Cache partition.